TP5 – Igel Ärgern

Magistère Maths-Info

Nicolas-Zhuang Lalung
Laure-Élise Tousch

14 octobre 2026

Vue d'ensemble du devoir

Table des matières :

1. Introduction 2
2. Structure du projet 2
   2.1. `board.c` et `board.h` 2
   2.2. `input.c` et `input.h` 2
   2.3. `main.c` 2
3. Extensions 2
   3.1. Implémentation et utilisation 2
   3.2. Le hérisson dopé 3
   3.3. Les sandwichs 3
4. Synthèse 3
Bibliographie 3

Informations :

- **Cours :** SYS
- **Enseignant :** Martin Quinson
- **Chargé de TD :** Simon Rokicki
- **Date de rendu :** 14 octobre 2026
- **Temps de travail:** ~14 heures par personne





Introduction

Nous avons implémenté le jeu Igel Ärgern, un jeu de stratégie de type jeu de plateau, ainsi que deux extensions. Nous avons ainsi découpé le projet en 3 fichiers `.c`, gérant chacun un aspect différent du jeu, pour une organisation plus claire et une meilleure répartition des tâches initiales.

Structure du projet

`board.c` et `board.h`

Ces fichiers gèrent la structure du plateau elle-même, ainsi que les fonctions la manipulant, permettant dans les autres fichiers de coder en ignorant le plus possible les spécificités de la structure.

C'est également au début de ce fichier que se trouvent les directives `#define` permettant de configurer la taille du plateau ainsi que le nombre de joueurs / hérissons.

`input.c` et `input.h`

Ces fichiers servent à demander des entrées à l'utilisateur de manière sécurisée[1], pour éviter que le code plante sur une mauvaise entrée.

`main.c`

Ce fichier contient la boucle de jeu, et constitue donc la colle qui réunit le reste.

Extensions

Implémentation et utilisation

Comme il n'y a que peu d'extensions, nous n'avons pas décidé d'effectuer de réels découpage au niveau du code. Notamment, il n'y a pas eu de nouveau fichier ou d'ajout de structure particulière, les méthodes nécessaires liées à la gestion du plateau de jeu ont été ajoutées dans `board.c`, et les éventuelles variables nécessaires à l'activation des extensions et leur utilisation dans la boucle de jeu ont été ajoutées au début de la fonction `main` avec un préfixe `ext_`.

Il semblerait préférable cependant en cas d'ajout de plusieurs autres extensions et de complexification du code d'effectuer un découpage plus clair, par exemple avec des tructures ou des fichiers séparés.

Chaque extension a un code d'activation constitué d'un caractère. Afin d'activer une extension, on lance le jeu en spécifiant le ou les codes d'extensions en premier argument, attachés dans le cas où il y aurait plusieurs extensions.

3.1.1. Exemples d'utilisation

```
$ ./igel-argern # sans extensions
$ ./igel-argern d # une seule extension, de code d
$ ./igel-argern ds # plusieurs extensions, de codes respectifs d et s
```

Le hérisson dopé

Cette extension a pour code d'activation `d`.

La *variante du hérisson dopé*[2] permet de ne pas compter le premier héisson à atteindre son objectif, car il était *bien évidemment* dopé. Le joueur à qui il appartenait est donc obligé d'utiliser l'entièreté de ses hérissons restants pour pouvoir gagner. Ceci permet d'ajouter un peu de complexité à la stratégie.

L'implémentation de cette extension était assez simple, car il suffisait simplement de garder en mémoire le fait que le hérisson dopé n'avait pas encore franchi la ligne d'arrivée, de vérifier à chaque franchissement cette information, et d'éventuellement l'enregistrer tout en évitant de comptabiliser le hérisson en question si ce n'avait pas encore été fait.

Les sandwichs

Cette extension a pour code d'activation `s`.

La *variante des sandwichs*[2] permet de donner plus de puissance à un joueur qui possède plusieurs hérissons sur une case (dont celle en haut de la pile). En effet, il peut auquel cas décider de déplacer toute la section entre le hérisson du haut et le prochain hérisson lui appartenant (inclus). Cela concerne à la fois les déplacements verticaux et le long d'une ligne. Il ne peut pas déplacer plus de 2 hérissons lui appartenant à la fois en réalisant un sandwich.

Cette extension était plus dure à implémenter. Il était avant tout nécessaire d'ajouter deux fonctions à `board.c` afin d'indiquer la présence et la taille d'un « sandwich » sur une case, mais aussi afin de déplacer un éventuel sandwich d'une case à l'autre. Côté logique de jeu, cette extension rend la boucle de jeu un peu plus longue et fastidieuse à écrire, mais la difficulté principale résidait dans le fait que cela permet à un nombre arbitraire de hérissons de franchir la ligne d'arrivée lors de la même action.

3.1.1. Compatibilité avec la *variante du hérisson dopé*

Il a été décidé que lors de l'activation des deux extensions, si un sandwich franchit la ligne d'arrivée en premier, alors seulement le premier hérisson est pénalisé, à savoir celui du joueur ayant réalisé l'action.

Synthèse

Nous avons observé que la plus grande difficulté ici était le travail en équipe et la communication. En effet, Chaque membre de l'équipe doit comprendre exactement ce qu'il doit faire dans le cadre du travail, mais aussi du fait que quelqu'un d'autre travaille au même projet.

Bibliographie

---

[1] M. Nebra, « Apprenez à programmer en C ». [En ligne]. Disponible sur: https://web.archive.org/web/20220422165131/https://openclassrooms.com/fr/courses/19980-apprenez-a-programmer-en-c/16993-securisez-la-saisie-de-texte

[2] F. Nestel et D. Matthaus, « Hedgehogs in a Hurry (Igel Argern) variants ». [En ligne]. Disponible sur: http://gamecabinet.com/rules/IgelArgernVariants.html